



ROOT CAUSE ANALYSIS

VIRTUAL CONTROL

VMWARE CLOUD FOUNDATION SOLUTIONS

vCenter Redeployment & Recovery RCA

Full play-by-play of the 2026-05-13 vCenter loss, OVA-based redeployment via Host Client, SDDC Manager fsck recovery, cluster reconstruction, and VDS re-import — with every issue catalogued and every command annotated with its execution context.

VCENTER

VCSA RECOVERY

HOST CLIENT OVA

FSCK

VDS RE-IMPORT

LESSONS LEARNED

VCF 9.0.1

VMware Cloud Foundation

VCF 9 vCenter Redeployment & Recovery — Root Cause Analysis

Version: 1.0 | **Date prepared:** May 13, 2026 | **Author:** Virtual Control LLC | **Classification:** Internal — Operational

Incident Classification

Field	Value
Incident date	2026-05-13
Severity	P2 — Management plane outage; data plane retained
Components impacted	vCenter Server (lost), SDDC Manager (boot failure), networking inventory (VDS unmanaged)
Components preserved	vSAN data plane (100% healthy throughout), ESXi cluster membership, NSX overlay, all VCF appliance data
Recovery type	Full vCenter redeploy via Host Client OVA + Photon OS fsck recovery + new VDS construction
Recovery outcome	Management plane operational at end of session; NSX vmk migration and VM NIC migration deferred
Customer-equivalent impact	If in production: vCenter UI unavailable ~10 hours, VM operations degraded (no fresh provisioning), automated lifecycle paused, vSAN datastore and running VMs unaffected throughout
Recovery duration	~10 hours including diagnosis, three failed approaches before success, and extensive documentation
Root cause class	Architectural — VCSA UI Installer 9.0 cannot deploy onto an existing vSAN datastore (KB 389238), compounded by stale certificates on the target host and an orphan VDS from the destroyed vCenter

This RCA captures the full sequence of events during the 2026-05-13 vCenter Server redeployment in the VCF 9.0.1 nested lab. The starting state was: vCenter Server `vcenter.lab.local` destroyed; ESXi hosts and vSAN cluster healthy at the data-plane level; VCF management appliances (SDDC Manager, Fleet/LCM, VCF Operations, NSX Manager) running but disconnected from management. The end state was: new vCenter deployed via Host Client OVA, datacenter and cluster recreated, all 4 hosts re-added with image-based management, new vSphere Distributed Switch (VDS) created and the management/vMotion/vSAN VMkernel adapters migrated to it, SDDC Manager recovered from a power-outage-induced filesystem corruption, and the cluster operationally green. NSX VMkernel adapter migration and VM NIC migration were deferred to a follow-up session.

Every command in this document is annotated with the terminal/shell where it was executed — see [Terminal Conventions](#) for the legend. Errors are quoted verbatim from logs or UI; analysis mistakes during the session are noted as such so the next operator does not repeat them.

Table of Contents

1. Executive Summary
2. Terminal Conventions
3. Environment Inventory
4. Initial State (Pre-Session)
5. Issue #1 — VCSA UI Installer Cannot Deploy onto Existing vSAN
6. Issue #2 — esxi04 SSL Certificate Issued by Destroyed vCenter VMCA
7. Issue #3 — vsanmgmt.d Cached Stale Cert After Regen, Installer Hung on GetVsanRuntimeInfo
8. Phase A — vCenter Deploy via Host Client OVA Upload
9. Phase B — Stage 2 (vCenter Server Setup)
10. Phase C — Datacenter + Cluster Creation
11. Phase D — Host Re-Add with Image Extraction
12. Issue #4 — SDDC Manager Stuck in systemd Emergency Mode
13. Issue #5 — Storage I/O Controller Not in VCG (Cosmetic)
14. Issue #6 — Soft CPU Lockup Diagnosis (Service Start Storms)
15. Phase E — VDS Re-Import
16. Issue #7 — `config.owner` Error During Add-and-Manage-Hosts Wizard
17. Issue #8 — MM Blocked by Running VMs
18. Issue #9 — Wizard Doesn't List Hosts to "Add"
19. SSH-Based vmk Migration (esxi01)
20. Per-Host VDS Migration via Wizard (esxi02/03/04)
21. End-of-Session State
22. Deferred Work
23. Lessons Learned / Mistakes Made
24. Commands Reference Index
25. Document Information

1. Executive Summary

Phase	Outcome	Duration (approx)
Pre-session vCenter UI Installer attempt	FAILED — KB 389238 limitation (VCSA UI Installer 9.0 cannot use existing vSAN)	~3 hours including debug
Host Client OVA deploy of VCSA 9.0.1 onto esxi04	SUCCESS	~30 min upload + ~25 min firstboot
Stage 2 (VAMI wizard)	SUCCESS	~15 min
Datacenter + Cluster creation in new vCenter	SUCCESS	5 min
Host re-add with image extraction (4 hosts)	SUCCESS	~15 min
SDDC Manager boot recovery via GRUB <code>fsck.repair=yes</code>	SUCCESS	~10 min
VDS <code>vcenter-cl01-vds01</code> creation + 6 port groups	SUCCESS	15 min
Phase 3 wizard apply (first attempt, all 4 hosts)	FAILED — <code>config.owner</code> error from orphan-DVS removal blocked by 6 vmks attached	(caught and rolled back automatically)
SSH-based vmk surgery on esxi01 (vSwitch-tmp bridge)	SUCCESS	~25 min
Wizard re-attempt on esxi01 (after surgery)	SUCCESS for vmnic0 + vmk0	

Phase	Outcome	Duration (approx)
Wizard apply on esxi02 / esxi03 / esxi04 (with orphan still holding NSX vmks)	SUCCESS for vmk0 + vmnic0	
Migrate vmk1 + vmk2 to new VDS port groups via wizard (all 4 hosts)	SUCCESS	~10 min/host
vmnic1 → new VDS dvUplink2	DEFERRED to NSX vmk migration session	
NSX vmks (vmk10/11/50) migration	DEFERRED	
VM NIC migrations + HA re-enable + orphan VDS removal	DEFERRED	

Final cluster state at end of session: 4 nodes HEALTHY, 194 vSAN objects healthy, 0 inaccessible, 0 active-rebuild. Management, vMotion, and vSAN traffic on the new VDS `vcenter-cl01-vds01`. NSX overlay traffic continues on the orphan VDS `vcenter-cl01-vds01-new` via vmnic1 — this is the intended interim state until the NSX phase completes.

2. Terminal Conventions

Every code block in this document is annotated with where the command was executed:

Tag	Meaning
PowerShell @ workstation	Windows 11 workstation (192.168.1.160). Used for <code>plink</code> , <code>Test-Connection</code> , <code>Test-NetConnection</code> , <code>Resolve-DnsName</code> , <code>Start-Process</code> , certificate retrieval via <code>System.Net.Sockets.TcpClient</code> .
ESXi shell @ <host>	SSH session to an ESXi host. Used for <code>esxcli</code> , <code>vim-cmd</code> , <code>vsish</code> , direct <code>/sbin/generate-certificates</code> , <code>/etc/init.d/<service> restart</code> , <code>services.sh restart</code> . Authenticated via <code>plink -ssh -batch -hostkey "SHA256: ..." -pw '...' root@<ip></code> .
Host Client UI @ <host>	Per-ESXi web UI at <code>https://<host>/ui/</code> , logged in as <code>root</code> . Used when vCenter UI was unavailable.
vSphere Client UI	Main vCenter web UI at <code>https://vcenter.lab.local/ui</code> , logged in as <code>administrator@vsphere.local</code> .
VAMI @ vcenter:5480	vCenter Server Appliance Management Interface at <code>https://vcenter.lab.local:5480</code> . Stage 2 setup wizard ran here.
VM Console @ GRUB	VM console accessed via Host Client (for the affected appliance) at the GRUB boot menu.
VCSA UI Installer	Standalone Windows executable from the VCSA ISO at <code>D:\vcsa-ui-installer\win32\installer.exe</code> (drive letter varies with ISO mount).

3. Environment Inventory

Component	Identifier	Notes
Workstation	Dell Precision 7920, Windows 11 26200	192.168.1.160; PowerShell 5.1, plink (PuTTY 0.78), VMware Workstation host
ESXi 01	esxi01.lab.local	192.168.1.74; root pw <ESXI01_PW> (different from rest)
ESXi 02	esxi02.lab.local	192.168.1.75; root pw <LAB_PW>
ESXi 03	esxi03.lab.local	192.168.1.76; root pw <LAB_PW>
ESXi 04	esxi04.lab.local	192.168.1.82; root pw <LAB_PW>; rebuild target for new vCenter
vSAN	sub-cluster UUID 52b68472-c149-d706-5c7e-ef39e7136ffa	OSA, 4-node, vSAN datastore vcenter-cl01-ds-vsan01
Destroyed vCenter	vcenter.lab.local / 192.168.1.69	Will be rebuilt with same identity
VCF Ops	vcf-ops.lab.local / 192.168.1.77	Survived; running on esxi02
SDDC Manager	sddc-manager.lab.local	Survived but in systemd emergency mode (see Issue #4)
Fleet/LCM	fleet.lab.local / 192.168.1.95	Survived; running on esxi02
NSX Manager	nsx-mgr.lab.local / 192.168.1.70	Survived; running on esxi01 alone (dedicated host per policy)
DNS server	192.168.1.230	Lab DNS; resolves vcenter.lab.local ↔ 192.168.1.69 forward+reverse
NTP server	192.168.1.230	Same VM
VCSA ISO	I:\VCF-Depot\PROD\COMP\VCENTER\VMware-VCSA-all-9.0.1.0.24957454.iso	Mounted to various drive letters during session (D:, L:)

4. Initial State (Pre-Session)

Verified at session start:

```
# ESXi shell @ esxi04
esxcli vsan debug object health summary get
```

Output (truncated to relevant lines):

```
inaccessible                                0
reduced-availability-with-no-rebuild        0
reduced-availability-with-active-rebuild    0
data-move                                   0
healthy                                    141
```

```
# ESXi shell @ esxi04
esxcli vsan cluster get
```

Output (relevant lines):

```
Enabled: true
Local Node State: AGENT
Local Node Health State: HEALTHY
Sub-Cluster Member Count: 4
Sub-Cluster Member HostNames: esxi03.lab.local, esxi02.lab.local, esxi01.lab.local,
esxi04.lab.local
Sub-Cluster UUID: 52b68472-c149-d706-5c7e-ef39e7136ffa
vSAN ESA Enabled: false
```

Pre-conditions met: - vSAN cluster intact and healthy from the hosts' perspective (no vCenter required for vSAN data path) - All four hosts running ESXi 9.0.1 build 24957456 - DNS forward + reverse for `vcenter.lab.local` verified working via 192.168.1.230 - VCF management VMs in unknown state (most running but unmanaged)

5. Issue #1 — VCSA UI Installer Cannot Deploy onto Existing vSAN

5.1 Trigger

The `VCF9-Host-vCenter-Rebuild-Plan.md` runbook §6.2 (v1) directed: "Deploy a new vCenter Server appliance via the ISO installer UI, using the same IP, FQDN, and SSO domain name as the previous vCenter."

This was attempted with `D:\vcsa-ui-installer\win32\installer.exe`. The wizard reached the **Select Datastore** step (step 6) where only the vSAN datastore `vcenter-cl01-ds-vsan01` was visible.

5.2 Symptoms during the attempt

5.2.1 First wall: Stage 1 validation hung 10–15 minutes. While targeting esxi04 via the installer's "vCenter Server Deployment Target" step, the validating spinner ran indefinitely.

Diagnostic from the host:

```
# ESXi shell @ esxi04
grep -A 8 'SSLVerifyException' /var/log/hostd.log | tail -40
```

Surfaced:

```
2026-05-13T13:12:01.806Z Wa Hostd: Failed to get pooled connection;
... TCP:localhost.localdomain:80, SSL Exception: Verification parameters:
PeerThumbprint:      10:CC:67:EA:10:52:B4:27:49:AC:03:57:87:1B:72:48:E2:1F:A4:E3
ExpectedThumbprint: A2:79:49:74:45:A3:62:1E:A1:D9:A7:12:7E:E0:72:13:F6:AB:4B:59
PeerCertificate: <base64 with 0=vcenter.lab.local (destroyed VMCA)>
```

The hostd was trying to call `localhost:80 /vsanperf`, the peer presented a cert signed by the destroyed vCenter VMCA, and the expected thumbprint (post-regen) didn't match. This was Issue #3 (see §7).

5.2.2 Real root cause discovered later via Broadcom KB 389238: the VCSA UI Installer 9.0 cannot use an existing vSAN datastore for installation. Verbatim from the KB:

"the new vSAN cluster bootstrapping option in VCSA UI Installer 9.0 will NOT work, and the deployment process fails at 'Install - Stage 2: vCenter Server setup is in progress'."

"Use the option 'Install on an existing datastore accessible from the target host' to proceed with the VCSA deployment using VCSA UI Installer 9.0. After VCSA is deployed, create a vSAN cluster through VCSA's vSphere Client user interface."

"Cannot use existing vSAN datastore for installation" — KB 389238

5.3 Resolution

Abandoned the VCSA UI Installer entirely in favor of Host Client OVA upload (see §8). The Host Client OVA path does not invoke the same datastore validation pre-check and successfully deploys onto vSAN.

5.4 Memory + Documentation updates

- Created memory: `feedback_vcsa_use_host_client_ova` — the rule
- Created memory: `reference_vcsa_redeploy_ovf_properties` — verified OVF property values for the homelab
- Updated runbook `VCF9-Host-vCenter-Rebuild-Plan.md` §6.2 to direct readers to Host Client OVA + reference KB 389238
- Added Issue #42 to `VCF-Undocumented-Issues-Reference.md`

6. Issue #2 — esxi04 SSL Certificate Issued by Destroyed vCenter VMCA

6.1 Trigger

When the VCSA UI Installer attempted the "vCenter Server Deployment Target" step pointing at esxi04 (192.168.1.82), the wizard initially reported:

"Failed to get an SSL certificate of the target server."

6.2 Diagnostic — confirm cert is reachable, then inspect the cert chain

```
# PowerShell @ workstation
$tcp = New-Object System.Net.Sockets.TcpClient
$tcp.Connect("192.168.1.82", 443)
$ssl = New-Object System.Net.Security.SslStream($tcp.GetStream(), $false, ({ $true }))
$ssl.AuthenticateAsClient("192.168.1.82")
$cert = $ssl.RemoteCertificate
"Subject: $($cert.Subject)"
"Issuer: $($cert.Issuer)"
"NotBefore: $($cert.GetEffectiveDateString())"
$ssl.Dispose(); $tcp.Close()
```

Output:

```
Subject: E=vmca@vmware.com, CN=esxi04.lab.local, OU=VMware Engineering, O=VMware, L=Palo Alto, S=California, C=US
Issuer: OU=VMware Engineering, O=vcenter.lab.local, S=California, C=US, DC=local, DC=vsphere, CN=CA
NotBefore: 5/4/2026 6:29:41 PM
```

The **issuer** was the destroyed vCenter's VMCA (O=vcenter.lab.local, CN=CA). The installer attempted to validate the chain; the root was unreachable; cert fetch failed.

6.3 Resolution — Regenerate esxi04 Self-Signed Cert per KB 389767

```
# ESXi shell @ esxi04 (via plink)
mv /etc/vmware/ssl/rui.crt /etc/vmware/ssl/rui.crt.bak
mv /etc/vmware/ssl/rui.key /etc/vmware/ssl/rui.key.bak
/sbin/generate-certificates
ls -la /etc/vmware/ssl/rui.crt /etc/vmware/ssl/rui.key
openssl x509 -in /etc/vmware/ssl/rui.crt -noout -subject -issuer -dates
```

Verified new cert:

```
subject= ... CN = esxi04.lab.local ...
issuer=O = VMware Installer
notBefore=May 13 12:46:57 2026 GMT
notAfter=Nov 11 12:46:57 2037 GMT
```

Then restart services so they read the new cert from disk:

```
# ESXi shell @ esxi04
services.sh restart
```

This took ~1m 44s. The SSH session survived (some services in `services.sh` are restarted serially; SSH was restarted but reconnected automatically via plink retry behavior).

External verification:

```
# PowerShell @ workstation - confirm new cert is being served
$tcp = New-Object System.Net.Sockets.TcpClient
$tcp.Connect("192.168.1.82", 443)
$ssl = New-Object System.Net.Security.SslStream($tcp.GetStream(), $false, ({$true}))
$ssl.AuthenticateAsClient("192.168.1.82")
"Subject: $($cert.Subject)"
"Issuer: $($cert.Issuer)"
"NotBefore: $($cert.GetEffectiveDateString())"
```

```
Subject: OID.1.2.840.113549.1.9.2=" ... ", CN=esxi04.lab.local, ...
Issuer: O=VMware Installer
NotBefore: 5/13/2026 8:46:57 AM
```


New cert in production. **New SHA1 thumbprint:**

A2:79:49:74:45:A3:62:1E:A1:D9:A7:12:7E:E0:72:13:F6:AB:4B:59 — recorded for later "Accept Thumbprint" prompts during host re-add.

6.4 References

- KB 389767 — Unable to add ESXi host to vCenter due to error "Certificate unable to verify"
- KB 426701 — SSL Handshake Failed / Host name does not match

7. Issue #3 — vsanmgmtd Cached Stale Cert After Regen, Installer Hung on GetVsanRuntimeInfo

7.1 Trigger

After Issue #2's cert regen + `services.sh restart`, the VCSA UI Installer's Stage 1 validating spinner hung for 10–15 minutes with no progress reported in the wizard.

7.2 Diagnostic

Confirmed installer was actively talking to esxi04 (not network-stalled):

```
# PowerShell @ workstation
Get-NetTCPConnection -RemoteAddress 192.168.1.82 -State Established |
  Select-Object LocalPort,RemotePort,@{N='Proc';E={(Get-Process -Id
  $_.OwningProcess).ProcessName}}
```

Output showed 1 ESTABLISHED connection from `installer` to `192.168.1.82:443`. So the installer process was alive and connected, just waiting.

Inspecting hostd:

```
# ESXi shell @ esxi04
grep -A 8 'SSLVerifyException' /var/log/hostd.log | tail -40
```

The smoking gun (already quoted in §5.2.1):

```
Invoke method vim.host.VsanSystemEx.GetVsanRuntimeInfo failed:
SSL Exception: Verification parameters:
PeerThumbprint:      10:CC:67:EA:10:52:B4:27:49:AC:03:57:87:1B:72:48:E2:1F:A4:E3    (stale)
ExpectedThumbprint: A2:79:49:74:45:A3:62:1E:A1:D9:A7:12:7E:E0:72:13:F6:AB:4B:59    (new)
```

The `/vsanperf` endpoint is served by `vsanperfsvc` (part of `vsanmgmtd`). `services.sh restart` had restarted hostd and rhttpproxy but **did not re-initialize vsanmgmtd's in-memory cert cache** — `vsanperf` was still presenting the old cert; hostd's new expected thumbprint didn't match → SSL Verify Exception → `GetVsanRuntimeInfo` failed → installer stuck.

7.3 Resolution

```
# ESXi shell @ esxi04
/etc/init.d/vsanmgmt restart
# Output:
# Terminating vsanperfsvc with PID 2098750
# vsanperfsvc stopped.
# vsanperfsvc started.

# Verify the served cert externally
```

```
# PowerShell @ workstation
$tcp = New-Object System.Net.Sockets.TcpClient
$tcp.Connect("192.168.1.82", 443)
$ssl = New-Object System.Net.Security.SslStream($tcp.GetStream(), $false, ({$true}))
$ssl.AuthenticateAsClient("192.168.1.82")
"$($cert.Issuer)"
# Expected: O=VMware Installer (was the destroyed vCenter VMCA before)
```

Output confirmed new cert in service.

7.4 Side-effect — vSAN rebuild

After `services.sh restart` + `vsanmgmt restart`, vSAN saw esxi04 components briefly absent and started rebuilds. From an unaffected host (esxi02):

```
# ESXi shell @ esxi02
esxcli vsan debug object health summary get
```

Showed `reduced-availability-with-active-rebuild: 10` and `healthy: 131` (was 141). Cluster recovered fully ~5 minutes later. No data was inaccessible at any point.

7.5 Why this matters in the doc

This is a **non-obvious side-effect of certificate regeneration on a vSAN node**. Standard cert-regen procedures (KB 389767) don't mention vsanmgmt. Per the Undocumented Issues Reference (issue ID assigned in v6.1) this was added because Broadcom's KB doesn't surface it.

8. Phase A — vCenter Deploy via Host Client OVA Upload

8.1 Prerequisites

Item	Verified
esxi04 cert regenerated, services serving new cert	✓ (Issues #2 + #3 resolved)
vSAN healthy: 141 healthy, 0 inaccessible, 0 rebuilding	✓
DNS forward + reverse for <code>vcenter.lab.local</code> ↔ <code>192.168.1.69</code>	✓ via 192.168.1.230

Item	Verified
VCSA ISO mounted	PowerShell @ workstation ISO mounted to L:\
OVA file accessible	L:\vcsa\VMware-vCenter-Server-Appliance-9.0.1.0.24957454_OVF10.ova (8.85 GB)

8.2 Deploy procedure

Host Client UI @ esxi04 at <https://192.168.1.82/ui/> logged in as root / <LAB_PW>.

1. Left nav → **Virtual Machines** → **Create / Register VM**
2. **Select creation type:** Deploy a virtual machine from an OVF or OVA file → Next
3. **Enter name and select files:** VM name `vcenter` ; drag-and-drop the .ova file (or click-to-select)
4. **Select storage:** Datastore `vcenter-cl01-ds-vsan01` ; Disk provisioning **Thin** → Next
5. **License agreements:** accept → Next
6. **Deployment options:** - Network mappings → Network 1: `pg-mgmt-ephemeral` (ephemeral DVPG; survives vCenter outage) - Disk provisioning: Thin - Deployment type: `tiny` - Next
7. **Additional settings** — OVF properties as captured in `reference_vcsa_redeploy_ovf_properties.md` :

OVF Property	Value
Host Network IP Address Family	<code>ipv4</code>
Host Network Mode	<code>static</code>
Host Network IP Address	<code>192.168.1.69</code>
Host Network Prefix	<code>24</code>
Host Network Default Gateway	<code>192.168.1.1</code>
Host Network DNS Servers	<code>192.168.1.230</code>
Host Network Identity	<code>vcenter.lab.local</code>
Domain Name (Networking Properties)	<code>lab.local</code>
Domain Search Path	<code>lab.local</code>
SSO Domain Name	<code>vsphere.local</code>
SSO Site Name	<code>Default-Site</code>
SSO Directory Username (read-only)	<code>administrator</code>
SSO Directory Password	<i>operator chosen</i>
Root Password (System Config)	<i>operator chosen</i>
Enable SSH	<code>True</code>
NTP Servers	<code>192.168.1.230</code>
CEIP enabled	<code>False</code>
All Upgrade Configuration fields	LEAVE BLANK

8. Ready to complete → Finish

Upload took ~5 minutes (8.85 GB to vSAN). VM created and auto-powered-on per the wizard's default.

8.3 Diagnostic gotcha encountered

During first boot, `vim-cmd vmsvc/get.config 15 | grep vAppConfig` reported:

```
vAppConfig = (vim.vApp.VmConfigInfo) null
```

This **looked like** the OVF properties had been dropped — the same failure mode as the prior NSX 9 ovftool 5.0 incident documented in `reference_powercli_import_vapp_for_nsx9.md`. Brief panic ensued.

Reality: Host Client uses a different OVF environment transport (likely ISO-mounted via CD-ROM at first boot, or guestinfo.* ExtraConfig keys not surfaced in the `vAppConfig` field of `vim-cmd vmsvc/get.config`). The deploy actually succeeded. After the ~15–25 minute firstboot completed, the appliance came up at 192.168.1.69 with the correct hostname.

This finding was documented as **Issue #41** in the Undocumented Issues Reference and as a "Diagnostic gotcha" section in `reference_vcsa_redeploy_ovf_properties.md`.

8.4 Verification

```
# PowerShell @ workstation
Test-Connection -ComputerName vcenter.lab.local -Count 4
```

Once pinging cleanly, monitor for VAMI port:

```
while (-not (Test-NetConnection vcenter.lab.local -Port 5480 -WarningAction
SilentlyContinue).TcpTestSucceeded) {
    Start-Sleep 10
}
"Stage 2 reachable."
```

Browser confirmed: `https://vcenter.lab.local:5480` answered.

9. Phase B — Stage 2 (vCenter Server Setup)

VAMI @ vcenter:5480

1. **Welcome** → Set up appliance
2. **Configure appliance** — values matched what was set via OVF properties; review only
3. **SSO configuration:** - Single Sign-On Domain: **Create a new SSO domain** (the old SSO died with the destroyed vCenter; nothing to join) - Domain name: `vsphere.local` (same as old, for downstream identity continuity) - Username: `administrator` (fixed) - Password: as set in OVF - Site name: `Default-Site`
4. **CEIP:** unchecked
5. **Ready to complete:** Finish

Stage 2 completed in ~10 min. New vCenter UI reachable at `https://vcenter.lab.local/ui` shortly after.

10. Phase C — Datacenter + Cluster Creation

10.1 Datacenter

Right-click `vcenter.lab.local` → **New Datacenter** → Name: `vcenter-dc01` → OK.

10.2 Cluster

Right-click `vcenter-dc01` → **New Cluster**.

Setting	Value	Rationale
Name	<code>vcenter-cl01</code>	Matches old name and existing host-side VDS naming convention
Location	<code>vcenter-dc01</code>	
vSphere DRS	ON	Standard
vSphere HA	OFF	Per <code>[[feedback_disable_ha_before_uplink_migration]]</code> — enable after VDS is unified
vSAN	ON	Will inherit existing vSAN cluster identity from hosts
Enable vSAN ESA	unchecked	Verified existing cluster is OSA via <code>esxcli vsan cluster get (vSAN ESA Enabled: false)</code>
Manage configuration at cluster level	unchecked	
Image management	Extract an image from a new host → <code>esxi04.lab.local</code>	Image is canonical from esxi04 (recently regenerated cert, matches build 24957456)

EVC mode set to **Intel Skylake** per RCA §13.4 — matches prior cluster baseline.

10.3 vSAN configuration

vSAN HCI Mesh: No; Architecture: OSA; Dedup/Compression: Off; Encryption: Off — these match the existing host-side vSAN sub-cluster (`Sub-Cluster UUID 52b68472-c149-d706-5c7e-ef39e7136ffa`).

11. Phase D — Host Re-Add with Image Extraction

11.1 Image extraction

During New Cluster wizard "Image management" step, selected **Extract an image from a new host** → entered `esxi04.lab.local`, root, password. Accepted the new cert thumbprint (matched `SHA1 A2:79:49:74:45:A3:62:1E:A1:D9:A7:12:7E:E0:72:13:F6:AB:4B:59`). Extraction took ~30 seconds.

11.2 Host adds — order and credential gotcha

Right-click cluster `vcenter-cl01` → **Add Hosts**. Added each in turn:

Order	Host	Password used
1	esxi04.lab.local	<LAB_PW>
2	esxi02.lab.local	<LAB_PW>
3	esxi03.lab.local	<LAB_PW>
4	esxi01.lab.local	<ESXI01_PW> (different — flagged by [[reference_homelab_esxi_creds]])

Initially attempted esxi01 with the wrong password (the cluster-wide one, not esxi01's) → got "Failed to authenticate host 'esxi01.lab.local'." Resolved by switching to <ESXI01_PW>.

Each host added cleanly to the cluster. Cert thumbprint prompts accepted. vSAN cluster auto-discovered (each host already had the right Sub-Cluster UUID, so they joined into the new cluster object as the same vSAN cluster instance).

11.3 Post-add verification

```
# ESXi shell @ esxi04
esxcli vsan debug object health summary get
```

Showed **healthy: 160, inaccessible: 0** after all 4 added. The +19 from baseline 141 represents the new vCenter VM's vSAN objects (namespace + VMDKs + swap).

12. Issue #4 — SDDC Manager Stuck in systemd Emergency Mode

12.1 Trigger

After all 4 hosts were in the cluster, the user noticed **sddc-manager** VM (esxi03 vmid 3) in systemd emergency mode. Console screenshot showed:

```
[TIME]    Timed out waiting for device 6-9fc1-4f86-8019-aa93b37d37d0.
[DEPEND]  Dependency failed for /boot/efi
[DEPEND]  Dependency failed for Local File Systems
[TIME]    Timed out waiting for device d-ca74-4f95-9b9d-a5de5e4db99f.
[DEPEND]  Dependency failed for /boot
[FAILED]  Failed to start Rule-based for Device Events and Files
You are in emergency mode. ... Give root password for maintenance (or press Control-D to
continue):
```

Other VCF appliances (vcf-ops, fleet, collector, nsx-manager) were running but the user reported "soft CPU lockups" in the kernel logs.

12.2 Diagnostic — rule out vSAN

```
# ESXi shell @ esxi04
esxcli vsan debug object health summary get
```

Result: 194 healthy, 0 inaccessible. vSAN data plane fine. The boot failure was guest-OS-level, not infrastructure.

```
# ESXi shell @ esxi03 (where sddc-manager VM lives)
vim-cmd vmsvc/get.config 3 | grep -E 'fileName|scsi[0-9]+:[0-9]+\.\present'
```

All 6 VMDKs present and attached. The issue was inside the guest — partition UUIDs in `/etc/fstab` no longer matched. Most likely root cause: **partition GPT damage from the 2026-05-05 power outage** (recorded in `[[project_power_outage_20260505]]`).

12.3 Resolution — `fsck.repair=yes` via GRUB

VM Console @ GRUB on the sddc-manager VM:

1. From vSphere Client: right-click sddc-manager → Power → Reset
2. Open VM console immediately
3. At GRUB menu, press `e` to edit the default Photon kernel entry
4. Navigate to the line beginning with `linux` (kernel command line)
5. Append space then `fsck.repair=yes`
6. Press `Ctrl-X` to boot

Boot ran `systemd-fsck` against `/boot/efi`, `/boot`, and other ext4 filesystems. Multiple "FILE SYSTEM WAS MODIFIED" lines appeared. After ~5 min the `systemd` init resumed, services started, and the appliance reached the login prompt.

Verified:

```
# PowerShell @ workstation
Test-Connection -ComputerName sddc-manager.lab.local -Count 4
```

Pingable. Procedure documented in detail in `VCF-Photon-Boot-Recovery-fsck-Handbook.md`.

12.4 References

- KB 326323 — VCF appliance stuck on `/sysroot` mount fail; `fsck.repair=yes` recovery
- New handbook: `I:\VCF 9 Full documentation\VCF-Operations-Library\06-Troubleshooting\VCF-Photon-Boot-Recovery-fsck-Handbook.md`

13. Issue #5 — Storage I/O Controller Not in VCG (Cosmetic)

13.1 Symptom

Skyline Health alert: "The storage I/O controller is not listed in the VMware Compatibility Guide (VCG)." Listed controller: VMware Inc VMware Virtual SATA Controller, PCI ID `15ad,07e0`, on all 4 hosts.

13.2 Diagnostic

```
# ESXi shell @ esxi03
/bin/esxcli vsan debug controller list
```

Output (relevant lines):

```
Device Name: vmhba0
Device Display Name: VMware Inc VMware Virtual SATA Controller
Used By VSAN: true
PCI ID: 15ad/07e0/15ad/07e0
Driver Name: vmw_ahci
```

13.3 Resolution

No action required — known nested-lab false positive. PCI ID `15ad,07e0` is the VMware Virtual SATA Controller used by nested ESXi VMs. VMware does not certify virtual controllers in the VCG. The alert can be acknowledged/suppressed in Skyline Health.

14. Issue #6 — Soft CPU Lockup Diagnosis (Service Start Storms)

14.1 Symptom

User reported "all VMs have soft CPU locks except sddc-manager" after all 4 hosts re-added.

14.2 Initial (incorrect) hypothesis

My first hypothesis: misclassified-capacity-disk vSAN latency per [\[\[reference_homelab_vsan_misclassified_capacity\]\]](#).

14.3 Verification — disproved that hypothesis

```
# ESXi shell @ each host
esxcli vsan storage list | grep -E "Display Name:|Is SSD:|Is Capacity Tier:"
```

All 4 hosts: every disk reported `Is SSD: true`. The misclassification was already fixed during the May 9–11 disk-group rebuild work — memory was stale. Updated [\[\[reference_homelab_vsan_misclassified_capacity\]\]](#) to mark as RESOLVED.

14.4 Actual likely cause

```
# ESXi shell @ esxi02 (host with most VMs)
vim-cmd vmsvc/getallvms
# Each VM's uptime checked via:
vim-cmd vmsvc/get.summary <vmid> | grep bootTime
```

Boot times:

VM	Boot time	Uptime at session
nsx-manager	2026-05-06 16:28	7 days
fleet	2026-05-10 15:17	3 days
collector	2026-05-10 20:19	3 days

VM	Boot time	Uptime at session
vcf-ops	2026-05-10 16:08	3 days

All four were running for days **before** today's new vCenter was deployed. They had been making periodic retry calls to the destroyed vCenter for 3–7 days. The Linux kernel's `softlockup_watchdog` accumulated false-positive lockup warnings from each retry-attempt-then-time-out cycle.

14.5 Resolution

Sequential reboots in priority order (per `[[project_lcm_failed_deploy]]` — fleet LAST):

Order	VM	Method
1	collector	Restart Guest
2	vcf-ops	Restart Guest
3	nsx-manager	Restart Guest
4	fleet (lcm)	Restart Guest (last)

(SDDC Manager was already freshly rebooted from the fsck recovery — skipped.)

15. Phase E — VDS Re-Import

15.1 Why a new VDS was needed

The destroyed vCenter took its inventory database with it, including the VDS named `vcenter-cl01-vds01-new`. The hosts retained that VDS as a proxy switch at the data plane (all vmks, both uplinks attached, working), but the new vCenter had no inventory record of it. vCenter UI's Networking inventory was empty except for "TempMgmt" (an orphan portgroup picked up during host re-add) and "Virtual Private Clouds" (a vCenter built-in namespace).

vCenter cannot adopt an orphan VDS without it being "owned" — the host-side proxy carries `config.owner = <destroyed vCenter instance UUID>`. The supported path is to build a NEW VDS in the new vCenter and migrate hosts onto it.

15.2 Phase 1 — Create new VDS

vSphere Client UI

Networking → right-click `vcenter-dc01` → Distributed Switch → New Distributed Switch:

Setting	Value
Name	<code>vcenter-cl01-vds01</code> (deliberately different from the orphan <code>vcenter-cl01-vds01-new</code>)
Version	9.0.0
Number of uplinks	2
Uplink names	Uplink 1 , Uplink 2 (UI default; cosmetic difference from PowerCLI's <code>dvUplink1/2</code>)

Setting	Value
MTU	changed to 1700 post-creation via Edit Settings → Advanced (UI wizard defaulted to 1500; host vmnics are at MTU 1700)
Network I/O Control	Enabled (UI default — harmless when no shares configured)
Default port group	unchecked

15.3 Phase 2 — Create port groups

For each, right-click `vcenter-cl01-vds01` → Distributed Port Group → New:

Name	Port binding	Purpose
<code>pg-mgmt</code>	Static	host vmk0 mgmt
<code>pg-vmotion</code>	Static	host vmk1 vMotion
<code>pg-vsan</code>	Static	host vmk2 vSAN
<code>pg-overlay-tep</code>	Static	host vmk10 / vmk11 NSX TEPs
<code>pg-vm-mgmt</code>	Static	management VM workloads
<code>pg-mgmt-ephemeral</code>	Ephemeral - no binding	vCenter VM NIC migration bridge (per RCA §12.4 / §12.5)

vCenter also auto-created the uplink port group `vcenter-cl01-vds-DVUplinks-43` (system-managed; do not delete).

16. Issue #7 — `config.owner` Error During Add-and-Manage-Hosts Wizard

16.1 Symptom

vSphere Client UI Add and Manage Hosts wizard on the new VDS `vcenter-cl01-vds01`:

- Selected task: `Add hosts`
- Selected all 4 hosts
- Assigned vmnic0 to `Uplink 1`
- Assigned vmk0/1/2/10/11 to corresponding port groups

Clicked Finish. ~21 seconds later, **all 4 tasks failed** with identical message:

"A specified parameter was not correct: config.owner"

(One task per host: task-396, task-397, task-398, task-399.)

16.2 Diagnostic — `hostd.log` on `esxi01`

```
# ESXi shell @ esxi01
grep -B1 -A 20 'DvsApplyOperationFault\|Cannot remove DVS' /var/log/hostd.log | tail -40
```

Surfaced the real error:

```
2026-05-13T19:32:39 Hostsvc.DvsManager opID=4c91e9e3-2ea1 sid=520c9ba1:
  Cannot remove DVS [50 37 13 3f 96 e4 3e 93-1d f0 3b 01 d4 76 6a f8]:
  6 virtual NICs attached to it
```

That UUID matches the orphan VDS `vcenter-cl01-vds01-new` from RCA §13.2.1. The wizard's apply step had attempted to **remove the orphan VDS as part of consistency cleanup**. The orphan had 6 vmks (vmk0–vmk2 + vmk10/11/50) still attached → remove failed → whole transaction rolled back → vCenter UI translated the `DvsApplyOperationFault` into the misleading top-line "config.owner" message.

16.3 Resolution path (high level)

To make the wizard succeed: first move enough vmks OFF the orphan that "remove DVS" doesn't get triggered (or doesn't fail). For this lab, that meant moving vmk0/1/2 off the orphan (NSX vmks 10/11/50 are owned by NSX and can stay on orphan as the NSX overlay path).

For esxi01 the wizard would not list it as "addable" again (because the failed apply had partially registered it), so SSH-based vmk surgery was used (see §19). After that, the wizard was re-run on esxi01 with task `Manage host networking`, which succeeded. Same task succeeded on esxi02/03/04 directly.

17. Issue #8 — MM Blocked by Running VMs

17.1 Symptom

In parallel with Issue #7, the Tasks pane showed earlier failed tasks:

"The operation is not allowed in the current state. Cannot enter maintenance mode on a host running VMs."

These were `Move host to desired state and cluster` tasks triggered by vLCM (vSphere Lifecycle Manager) image-compliance auto-remediation.

17.2 Why

The cluster is image-managed (image extracted from esxi04 — see §11.1). vLCM checks every host against the image and tries to bring non-compliant hosts into compliance — which requires MM. MM requires VM evacuation. We had no working vMotion infrastructure (orphan VDS, no vMotion vmk path between hosts via the new VDS yet).

17.3 Resolution

Acknowledged as noise during the VDS work — the failures didn't break anything, just cluttered the Tasks pane. After Phase E vmk migration completed and vMotion was effectively working again via the new VDS, vLCM remediation would succeed if rerun. Not addressed in this session.

18. Issue #9 — Wizard Doesn't List Hosts to "Add"

18.1 Symptom

After Issue #7's failure rolled back, retrying the Add and Manage Hosts wizard with task `Add hosts` showed **no hosts available to add**.

18.2 Why

The earlier failed apply had registered each host with the new VDS as a proxy switch (no uplinks attached). From vCenter's view, those hosts were "already on the VDS" — so they were filtered out of the "Add hosts" candidate list.

18.3 Resolution

Change wizard task from `Add hosts` to `Manage host networking`. That task lists hosts that are ALREADY on the VDS, with the ability to reconfigure their uplinks and vmks. This was the correct task once the partial-registration state existed.

19. SSH-Based vmk Migration (esxi01)

After Issue #7, esxi01 was treated as the "first canary" — moving its vmks off the orphan via SSH (not vCenter API) to bypass the config.owner gate, so the wizard could succeed on subsequent hosts.

19.1 Phase A — pre-flight: verify vmnic2 reaches lab network

ESXi shell @ esxi01

```
# Create standard vSwitch
esxcli network vswitch standard add --vswitch-name=vSwitch-tmp

# Attach unused vmnic2 as uplink
esxcli network vswitch standard uplink add --uplink-name=vmnic2 --vswitch-name=vSwitch-tmp

# Create test port group
esxcli network vswitch standard portgroup add --portgroup-name=TestConnectivity --vswitch-name=vSwitch-tmp

# Add test vmk99 with IP in the mgmt subnet
esxcli network ip interface add --interface-name=vmk99 --portgroup-name=TestConnectivity
esxcli network ip interface ipv4 set --interface-name=vmk99 --type=static --ipv4=192.168.1.199 --netmask=255.255.255.0

# Verify
esxcli network ip interface ipv4 get | grep -E "vmk0|vmk99"
```

PowerShell @ workstation

```
Test-Connection -ComputerName 192.168.1.199 -Count 4
```

```
# Expect: replies in ~1-40 ms
```

Test passed (1–41 ms response). vmnic2 reachable on mgmt L2 — safe to route vmk0 through it.

19.2 Phase B — atomic move of vmk0 to TempMgmt (mgmt vmk)

The challenge: any "remove vmk + re-add" sequence will drop mgmt connectivity for a moment. If the second command runs in the same SSH session and the first command kills SSH, the second never executes → lockout.

Solution: write the move sequence to a file, run it via `nohup` so it survives SSH session death, then exit SSH cleanly.

ESXi shell @ esxi01

```
# Create TempMgmt port group
esxcli network vswitch standard portgroup add --portgroup-name=TempMgmt --vswitch-
name=vSwitch-tmp

# Write the move sequence to a file
cat > /tmp/move_vmk0.sh <<'SCRIPT'
#!/bin/sh
sleep 3
esxcli network ip interface remove --interface-name=vmk0
esxcli network ip interface add --interface-name=vmk0 --portgroup-name=TempMgmt
esxcli network ip interface ipv4 set --interface-name=vmk0 --type=static --ipv4=192.168.1.74 -
-netmask=255.255.255.0 --gateway=192.168.1.1
esxcli network ip interface tag add --interface-name=vmk0 --tagname=Management
esxcli network ip interface set --interface-name=vmk0 --enabled=true

date > /tmp/move_vmk0_result.log
esxcli network ip interface ipv4 get | grep vmk0 >> /tmp/move_vmk0_result.log
esxcli network ip interface tag get -i vmk0 >> /tmp/move_vmk0_result.log
SCRIPT
chmod +x /tmp/move_vmk0.sh

# Launch detached; exit SSH cleanly
nohup /tmp/move_vmk0.sh > /tmp/move_vmk0.out 2>&1 &
echo "PID: $!"
sleep 1
exit
```

PowerShell @ workstation

```
Start-Sleep -Seconds 8
Test-Connection -ComputerName 192.168.1.74 -Count 4
# Expect: replies, confirming vmk0 came up on new portgroup
```

Result: mgmt restored in <10 sec; pings 0–34 ms.

19.3 Phase B continued — atomic moves of vmk2 (vSAN) and vmk1 (Mgmt+VSAN dual-tag)

Same nohup pattern. Created `TempVsan` and `TempMixed` port groups on vSwitch-tmp first, then atomic move.

For vmk2:

```
# ESXi shell @ esxi01
esxcli network vswitch standard portgroup add --portgroup-name=TempVsan --vswitch-
name=vSwitch-tmp

cat > /tmp/move_vmk2.sh <<'SCRIPT'
#!/bin/sh
sleep 3
esxcli network ip interface remove --interface-name=vmk2
esxcli network ip interface add --interface-name=vmk2 --portgroup-name=TempVsan
esxcli network ip interface ipv4 set --interface-name=vmk2 --type=static --ipv4=192.168.11.121
--netmask=255.255.255.0
esxcli network ip interface tag add --interface-name=vmk2 --tagname=VSAN
esxcli network ip interface set --interface-name=vmk2 --enabled=true
SCRIPT
chmod +x /tmp/move_vmk2.sh
nohup /tmp/move_vmk2.sh > /tmp/move_vmk2.out 2>&1 &
```

Verification surprise: prior to move, vmk2's tag was confirmed as **VSAN** (matched expectation). After move, vSAN went into a transient rebuild (76 objects active-rebuild briefly) but recovered in <2 minutes to 194 healthy.

For vmk1 (the dual-tag): same pattern, port group **TempMixed**, restored BOTH **Management** and **VSAN** tags:

```
# inside the nohup script for vmk1
esxcli network ip interface tag add --interface-name=vmk1 --tagname=Management
esxcli network ip interface tag add --interface-name=vmk1 --tagname=VSAN
```

19.4 Mid-stream finding — vmk-to-role mapping was not what I assumed

```
# ESXi shell @ esxi01
for v in vmk0 vmk1 vmk2 vmk10 vmk11 vmk50; do
  echo === $v ===
  esxcli network ip interface ipv4 get | awk -v v=$v '$1==v'
  esxcli network ip interface tag get -i $v
done
```

Verified mapping on esxi01:

vmk	IP	Tags	Inferred role
vmk0	192.168.1.74	Management	Mgmt
vmk1	192.168.12.121	Management, VSAN	Mgmt+vSAN failover
vmk2	192.168.11.121	VSAN	vSAN
vmk10	192.168.13.105	(none)	NSX TEP
vmk11	192.168.13.108	(none)	NSX TEP
vmk50	169.254.1.1	(none)	NSX hyperbus

Notable: vmk1 carries BOTH Management and VSAN tags (redundancy). vmk10/11/50 have NO tags — they are NSX-managed via OpaqueNetwork mechanisms, not standard vmk tags.

19.5 Final esxi01 state after surgery

- vmk0 → vSwitch-tmp / TempMgmt
- vmk1 → vSwitch-tmp / TempMixed
- vmk2 → vSwitch-tmp / TempVsan
- vmk10, vmk11, vmk50 → orphan VDS (untouched)
- vmnic0 → orphan VDS uplink
- vmnic1 → orphan VDS uplink
- vmnic2 → vSwitch-tmp uplink

vSAN: 194 healthy, 0 issues. Mgmt + SSH reachable on 192.168.1.74.

20. Per-Host VDS Migration via Wizard (esxi02/03/04)

20.1 Wizard sequence (per host)

vSphere Client UI

1. Networking → right-click `vcenter-cl01-vds01` → Add and Manage Hosts
2. Task: **Manage host networking** → Next
3. Select the single target host → Next
4. **Manage physical adapters:** assign vmnic0 → **Uplink 1** (leave vmnic1 alone) → Next
5. **Manage VMkernel adapters:** - vmk0 → **pg-mgmt** - (Initially only vmk0; vmk1 + vmk2 handled in a second pass per host) - Leave NSX vmks (vmk10/11/50) UNCHANGED - Next
6. **Migrate VM networking:** unchecked → Next
7. Finish

Result on each host: ☒ vmnic0 + vmk0 on new VDS; vmnic1 + remaining vmks on orphan.

20.2 Pass 2 — vmk1 + vmk2 migration per host

Same wizard:

- vmk1 → **pg-vmotion** (matches 192.168.12.x subnet semantic)
- vmk2 → **pg-vsan**
- vmk0, vmk10/11/50 — leave alone
- vmnic adapter step: skip

Result: ☒ All non-NSX vmks on new VDS.

20.3 Per-host verification between iterations

```
# ESXi shell @ esxi04 (uninvolved cluster member; gives a steady view)
esxcli vsan debug object health summary get | grep -E "inaccessible|healthy|rebuild|data-move"
```

Expected after each migration: `inaccessible 0, active-rebuild 0, healthy 194`. Brief rebuilds were observed during esxi01's surgery; subsequent UI-driven migrations on esxi02/03/04 caused **zero** rebuild events (smoother because vmnic1 stayed on orphan providing a continuous fallback path).

20.4 Analysis mistake during this phase (recorded for future operators)

When auditing post-migration state with a simplified script that parsed `Portset:` numbers:

```
# ESXi shell @ each host (the script I used)
esxcli network ip interface list | grep -E "Name:|Portset:"
```

I assumed `DvsPortset-0` was always the orphan and `DvsPortset-1` was always the new VDS. **That assumption is wrong.** The portset numbering is host-specific — assigned by hostd in the order each VDS was registered. On esxi03 it was flipped: `DvsPortset-0` was the new VDS. I incorrectly told the user "esxi03 vmk1 still on orphan" and they retried the wizard for nothing. The correct query keys on the VDS NAME, not the portset number:

```
# ESXi shell @ any host - DEFINITIVE per-vmk VDS mapping
esxcli network ip interface list | awk '
/^  Name: vmk/{vmk=$2}
/^  VDS Name:/{vds=$3}
/^$/ && vmk != "" {printf " %-7s → VDS=%s\n", vmk, vds; vmk=""; vds="(none)"}
'
```

Use VDS name, not portset number. Recorded in [Lessons Learned](#).

21. End-of-Session State

21.1 Per-host VDS assignments at session end

Host	vmk0	vmk1	vmk2	vmk10	vmk11	vmk50
esxi01	vcenter-cl01-vds01 (new)	vcenter-cl01-vds01	vcenter-cl01-vds01	vcenter-cl01-vds01-new (orphan)	orphan	orphan
esxi02	new	new	new	orphan	orphan	orphan
esxi03	new	new	new	orphan	orphan	orphan
esxi04	new	new	new	orphan	orphan	orphan

21.2 Uplinks at session end

Host	vmnic0	vmnic1	vmnic2	vmnic3
esxi01	new VDS Uplink 1	orphan VDS	vSwitch-tmp (mgmt bridge — still in place)	unused
esxi02	new VDS Uplink 1	orphan VDS	vSwitchTemp (old May 3 RCA bridge — still present, harmless)	unused
esxi03	new VDS Uplink 1	orphan VDS	unused	unused
esxi04	new VDS Uplink 1	orphan VDS	unused	unused

21.3 vSAN


```

inaccessible: 0
reduced-availability-with-rebuild: 0
data-move: 0
healthy: 194
Sub-Cluster Member Count: 4
Sub-Cluster Member HostNames: esxi03.lab.local, esxi04.lab.local, esxi02.lab.local,
esxi01.lab.local

```

21.4 Management plane VMs

VM	Host	Power	Reachable
vcenter (192.168.1.69)	esxi04 vmid 15	On	✓ UI at /ui, VAMI at :5480
sddc-manager (192.168.1.92)	esxi03 vmid 3	On (recovered via fsck)	✓
vcf-ops (192.168.1.77)	esxi02 vmid 7	On	✓
fleet (192.168.1.95)	esxi02 vmid 4	On	✓
collector (192.168.1.78)	esxi02 vmid 6	On	✓
nsx-manager (192.168.1.70)	esxi01 vmid 19	On	✓

21.5 Cluster services

- DRS: enabled
- vSphere HA: **disabled** (intentional — re-enable after VDS unification complete)
- vSAN: enabled (vCenter inherited host-side vSAN identity)
- EVC mode: Intel Skylake

22. Deferred Work

In priority order:

#	Item	Why deferred	Reference
1	Migrate NSX vmks (vmk10/11/50) from orphan VDS to new VDS	Needs NSX Compute Manager re-registration + Transport Node Profile refresh; raw esxcli moves risk breaking NSX overlay	RCA §13.3; KB 322036 / 372076 / 399868
2	Migrate vmnic1 from orphan VDS to new VDS Uplink 2	Cannot remove orphan while NSX vmks remain attached; vmnic1 is the orphan's only uplink and carries NSX overlay	Must follow #1
3	Remove orphan VDS vcenter-cl01-vds01-new proxy from each host	Requires zero vmks on orphan	Must follow #1
4	Migrate esxi01's vmk1 + vmk2 from vSwitch-tmp (TempMixed/TempVsan) to new VDS port groups	Standard switch was a bridge during Phase E surgery; cleanup pending	Via UI: easy once everything else is on new VDS
5	Remove vSwitch-tmp + vmk99 test interface on esxi01	Bridge cleanup	After #4

#	Item	Why deferred	Reference
6	Remove old <code>vSwitchTemp</code> on esxi02 (orphan from May 3 RCA recovery)	Pre-existing artifact	Independent
7	Migrate VM NICs from orphan/ephemeral port groups to new VDS port groups (especially vCenter's own NIC)	vCenter VM still on <code>pg-mgmt-ephemeral</code> (the ephemeral bridge); needs migration via Host Client (ephemeral → ephemeral → static dance per RCA §12.5)	Requires brief vCenter outage
8	Re-enable vSphere HA on cluster	Wait until VDS is unified and vMotion fully functional	Last
9	vLCM compliance remediation cycle	Will succeed once #1–#3 done	Auto

23. Lessons Learned / Mistakes Made

23.1 Things to do differently next time

#	Mistake	What to do instead
1	Assumed VCSA UI Installer would deploy onto vSAN	Always check Broadcom KB 389238 before launching VCSA UI Installer; default to Host Client OVA upload for any vSAN-only target
2	Restarted only <code>hostd/rhttpproxy</code> after cert regen on a vSAN node	Also restart <code>vsanmgmt</code> (<code>/etc/init.d/vsanmgmt restart</code>) after cert regen on any host that runs vSAN — <code>services.sh restart</code> doesn't fully refresh <code>vsanmgmt</code> 's cert cache
3	Diagnosed <code>vAppConfig=null</code> as failure after Host Client OVA deploy	Wait the full firstboot window (15–25 min) and verify the appliance pings on its configured IP. <code>vAppConfig=null</code> is normal for Host-Client-deployed OVAs because Host Client uses a different OVF environment transport
4	Read <code>Portset:</code> number to infer which VDS a vmk was on	Always read <code>VDS Name:</code> directly — Portset numbers are host-specific and can be flipped relative to other hosts
5	Trusted a 6-day-old memory about misclassified vSAN capacity disks	Verify live state before recommending a procedure based on memory; memory carries point-in-time observations and may be superseded
6	Tried to recommend Skyline "Remediate Inconsistent Configuration" for missing VDS	Read the documented direction of remediation carefully. "vCenter state is authoritative" pushes vCenter's view DOWN to hosts — wrong direction for missing-VDS recovery. The right path is build new VDS + migrate hosts, not import-orphan

23.2 Things that worked well

- The `nohup` + delayed `sleep` + atomic-script pattern for SSH-based vmk moves preserved mgmt connectivity across the brief remove/add window. Mgmt was restored in <10s with no manual recovery needed.
- Creating a test vmk99 on the target vmnic BEFORE moving vmk0 (mgmt) gave a recovery path. Never used; not needed; but its existence enabled confident execution.
- Using `pg-mgmt-ephemeral` (ephemeral binding) for the initial vCenter VM NIC was forward-thinking — when the VM NIC migration phase eventually runs, ephemeral source means the vCenter UI can migrate the NIC FROM it (whereas migrating FROM a static-binding orphan dvport is blocked, per RCA §12.4).
- Keeping vmnic1 on the orphan VDS as a continuous fallback path for vmk0 during vmnic0's transition meant only 1 of 2 uplinks was in motion at any time. After vmk0 was on the new VDS via vmnic0, vSAN/vmotion

vmks could be moved without any uplink-side risk.

24. Commands Reference Index

24.1 By topic

vSAN health:

```
# ESXi shell @ any host
esxcli vsan debug object health summary get
esxcli vsan debug resync summary get
esxcli vsan cluster get
esxcli vsan storage list
```

ESXi cert regen + service refresh:

```
# ESXi shell @ target host
mv /etc/vmware/ssl/rui.crt /etc/vmware/ssl/rui.crt.bak
mv /etc/vmware/ssl/rui.key /etc/vmware/ssl/rui.key.bak
/sbin/generate-certificates
services.sh restart
/etc/init.d/vsanmgmt restart # critical extra step on vSAN nodes
```

Cert verification from outside:

```
# PowerShell @ workstation
$tcp = New-Object System.Net.Sockets.TcpClient
$tcp.Connect("<host-ip>", 443)
$ssl = New-Object System.Net.Security.SslStream($tcp.GetStream(), $false, ({ $true }))
$ssl.AuthenticateAsClient("<host-ip>")
"Subject: $($ssl.RemoteCertificate.Subject)"
"Issuer: $($ssl.RemoteCertificate.Issuer)"
$ssl.Dispose(); $tcp.Close()
```

plink invocation pattern (host-key cached or pre-supplied):

```
# PowerShell @ workstation
plink -ssh -batch -hostkey "SHA256:<fingerprint>" -pw '<password>' -m <local-script-file>
root@<ip>
```

vmk audit (correct method, by VDS name):

```
# ESXi shell @ any host
esxcli network ip interface list | awk '
/^  Name: vmk/{vmk=$2}
/^  VDS Name:/{vds=$3}
```

```
/^$/ && vmk ≠ "" {printf " %-7s → VDS=%s\n", vmk, vds; vmk=""; vds="(none)"}
,
```

vmk tag inspection:

```
# ESXi shell @ any host
esxcli network ip interface tag get -i vmk<N>
```

Atomic vmk move (delete-then-add via nohup, preserves IP):

```
# ESXi shell @ host (substitute IP, mask, tag, port group)
cat > /tmp/move_vmk.sh <<'SCRIPT'
#!/bin/sh
sleep 3
esxcli network ip interface remove --interface-name=vmk<N>
esxcli network ip interface add --interface-name=vmk<N> --portgroup-name=<target-pg>
esxcli network ip interface ipv4 set --interface-name=vmk<N> --type=static --ipv4=<ip> --
netmask=<mask>
esxcli network ip interface tag add --interface-name=vmk<N> --tagname=<tag>
esxcli network ip interface set --interface-name=vmk<N> --enabled=true
SCRIPT
chmod +x /tmp/move_vmk.sh
nohup /tmp/move_vmk.sh > /tmp/move_vmk.out 2>&1 &
sleep 1
exit
```

VM identification + config inspection:

```
# ESXi shell @ host
vim-cmd vmshvc/getallvms
vim-cmd vmshvc/get.summary <vmid>
vim-cmd vmshvc/get.config <vmid>
vim-cmd vmshvc/power.getstate <vmid>
```

SDDC Manager / Photon OS boot recovery:

```
# VM Console @ GRUB on the affected appliance
1. Reset VM via vSphere Client → Power → Reset
2. At GRUB menu: press 'e' to edit
3. On the 'linux' line, append: fsck.repair=yes
4. Ctrl-X to boot
5. Wait for systemd-fsck to complete each filesystem; appliance reaches login when done
```

24.2 By execution context

Action	Terminal
OVA upload	Host Client UI @ target ESXi
Stage 2 VCSA setup	VAMI @ vcenter:5480

Action	Terminal
Datacenter / cluster / VDS create	vSphere Client UI
Host re-add	vSphere Client UI
vmk migration via wizard	vSphere Client UI
vmk migration via esxcli	ESXi shell @ host via plink
Cert regen	ESXi shell @ host
Cert verification	PowerShell @ workstation
fsck via GRUB	VM Console @ GRUB
vSAN health	ESXi shell @ any host

25. Document Information

Field	Value
Document Title	VCF 9 vCenter Redeployment & Recovery — Session Log
Version	1.0
Author	Virtual Control LLC
Date Created	May 13, 2026
Environment	VMware Cloud Foundation 9.0.1 / Nested Lab (4-host vSAN OSA)
Session start	2026-05-13 morning
Session end	2026-05-13 evening
Trigger	vCenter Server <code>vcenter.lab.local</code> lost (preceding incident); cluster intact at host level
Outcome	vCenter redeployed via Host Client OVA; cluster + datacenter recreated; new VDS unified for mgmt/vMotion/vSAN; NSX overlay deferred; vSAN 100% healthy throughout
Issues catalogued	9 (Issues #1–#9 in this document); 2 of which are also recorded in <code>VCF-Undocumented-Issues-Reference.md</code> as Issues #41–#42
Companion documents	<code>VCF-Photon-Boot-Recovery-fsck-Handbook.md</code> , <code>VCF-VDS-Reimport-Runbook-2026-05-13.md</code> , <code>VCF-Undocumented-Issues-Reference.md</code> , <code>VCF9-Host-vCenter-Rebuild-Plan.md</code> (\$6.2 corrected per Issue #1), <code>VCF9-Management-Plane-Loss-RCA-2026-05-01.md</code>
Memory updates	Created: <code>feedback_vcsa_use_host_client_ova</code> , <code>reference_vcsa_redeploy_ovf_properties</code> ; Updated: <code>reference_homelab_vsan_misclassified_capacity</code> (marked RESOLVED)

Revision History

Version	Date	Change
1.0	May 13, 2026	Initial release. Session log captured immediately after session end. All commands and outcomes verified during execution.

This document is part of the VCF 9.0 Lab Documentation Suite: - **VCF9-Master-Bible.pdf** — Complete VCF 9.0 reference guide - **VCF9-Host-vCenter-Rebuild-Plan.pdf** — Rebuild plan (\$6.2 updated per this session) - **VCF9-Management-Plane-Loss-RCA-2026-05-01.pdf** — Prior incident RCA (referenced \$7.3, \$12, \$13.2) - **VCF-Photon-Boot-Recovery-fsck-Handbook.pdf** — fsck recovery handbook (created during this session) - **VCF-VDS-Reimport-Runbook-2026-05-13.pdf** — VDS re-import runbook (created during this session) - **VCF-Undocumented-Issues-Reference.pdf** — Catalog of undocumented issues (Issues #41–#42 added during this session)

(c) 2026 Virtual Control LLC. All rights reserved.